

# Criação de **CRUD** em C#

Pedro Assenção

# Problemas

1

**Informações  
espalhadas em  
papéis ou  
planilhas**

2

Dificuldade para  
encontrar dados  
rapidamente

3

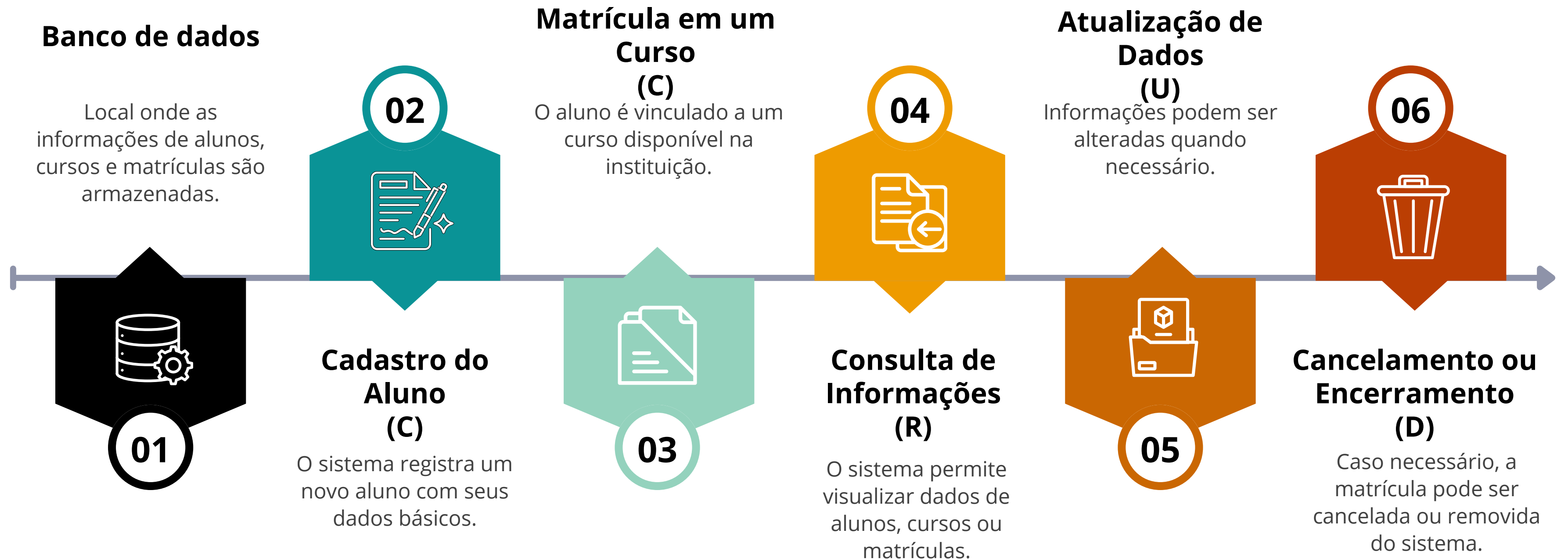
Risco de erros ao  
atualizar  
informações

# Objetivo

- **Centralizar informações em um único sistema**
- **Facilitar a busca e consulta de dados**
- **Reduzir erros na atualização de informações**
- **Aumentar a produtividade dos usuários**

# FLUXO CRUD

## SISTEMA ACADÊMICO



**Na prática**



# Banco de Dados

# Aplicação

## Tabela: Curso



## Classe: Curso

```
CREATE TABLE Curso (  
    IdCurso INT IDENTITY(1,1) PRIMARY KEY,  
    Nome VARCHAR(100) NOT NULL,  
    DuracaoSemestres INT NOT NULL  
);  
  
CREATE TABLE Aluno (  
    IdAluno INT IDENTITY(1,1) PRIMARY KEY,  
    Nome VARCHAR(150) NOT NULL,  
    Email VARCHAR(150),  
    DataNascimento DATE  
);  
  
CREATE TABLE Matricula (  
    IdMatricula INT IDENTITY(1,1) PRIMARY KEY,  
    IdAluno INT NOT NULL,  
    IdCurso INT NOT NULL,  
    DataMatricula DATE NOT NULL,  
    Status int DEFAULT 0,  
  
    FOREIGN KEY (IdAluno) REFERENCES Aluno(IdAluno),  
    FOREIGN KEY (IdCurso) REFERENCES Curso(IdCurso)  
);
```

```
namespace SistemaAcademico.API.Models  
{  
    using System;  
    using System.Collections.Generic;  
  
    9 references  
    public partial class Curso  
    {  
        0 references  
        public Curso()  
        {  
            this.Matricula = new HashSet<Matricula>();  
        }  
  
        0 references  
        public int IdCurso { get; set; }  
        0 references  
        public string Nome { get; set; }  
        0 references  
        public int DuracaoSemestres { get; set; }  
  
        1 reference  
        public virtual ICollection<Matricula> Matricula { get; set; }  
    }  
}
```

# Operações do sistema - Criar e Consultar

Consultar informações existentes

```
// GET: Cursos
0 references
public async Task<ActionResult> Index()
{
    return View(await db.Curso.ToListAsync());
}
```

Criar novos registros

```
[HttpPost]
[ValidateAntiForgeryToken]
0 references
public async Task<ActionResult> Create([Bind(Include = "IdCurso, Nome, DuracaoSemest
{
    if (ModelState.IsValid)
    {
        db.Curso.Add(curso);
        await db.SaveChangesAsync();
        return RedirectToAction("Index");
    }

    return View(curso);
}
```

# Operações do sistema - Atualizar e Remover

## Atualizar dados existentes

```
[HttpPost]
[ValidateAntiForgeryToken]
0 references
public async Task<ActionResult> Edit([Bind(Include = "IdCurso,Nome,DuracaoSemestres
{
    if (ModelState.IsValid)
    {
        db.Entry(curso).State = EntityState.Modified;
        await db.SaveChangesAsync();
        return RedirectToAction("Index");
    }
    return View(curso);
}
```

## Remover registros do sistema

```
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
0 references
public async Task<ActionResult> DeleteConfirmed(int id)
{
    Curso curso = await db.Curso.FindAsync(id);
    db.Curso.Remove(curso);
    await db.SaveChangesAsync();
    return RedirectToAction("Index");
}
```



# REFERÊNCIAS DA AULA

1. <https://learn.microsoft.com/pt-br/aspnet/core/data/ef-mvc/crud?view=aspnetcore-10.0>
2. <https://learn.microsoft.com/pt-br/aspnet/core/data/ef-mvc/migrations?view=aspnetcore-10.0>
3. <https://learn.microsoft.com/pt-br/aspnet/core/data/ef-mvc/read-related-data?view=aspnetcore-10.0>
4. <https://marcionizzola.medium.com/criando-um-simples-crud-com-net-mvc-com-dotnet-6-917cf723815e>